

BridgeIQ

AI-Powered Business Intelligence & Process Transformation

Complete Project Guide -- Every Feature, Every Decision, Every Line Explained



Built by: Sai Hemanth | Technical Business Analyst | 2026
gtrhemanth14@gmail.com | github.com/gtrhemanth/BridgeIQ-
Live at: <https://bridgeiq.streamlit.app>

Table of Contents

1. Project Overview & Purpose
2. The Business Scenario -- Apex Solutions
3. Technology Stack
4. Project File Structure
5. Data Layer -- 6 Tables & Schema
6. SQL Analytics Layer -- 10 Queries
7. App Architecture -- How Streamlit Works
8. Page 1 -- Home
9. Page 2 -- Executive Dashboard (9 Tabs)
10. Page 3 -- AI Insights Engine
11. Page 4 -- AI Feedback Analyzer
12. Page 5 -- AI Requirements Generator
13. Page 6 -- Interview Simulator
14. Page 7 -- Customer 360
15. Page 8 -- BA Artifacts
16. Page 9 -- What-If Revenue Simulator
17. Page 10 -- Churn Predictor (ML Model)
18. Page 11 -- SQL Playground
19. Page 12 -- Engagement Summary
20. AI Integration -- How Claude API Works
21. Navigation & Deep Links
22. CSS & Visual Design System
23. BA Deliverables Explained
24. How to Run Locally
25. How to Deploy on Streamlit Cloud
26. Key Design Decisions & Why
27. Frequently Asked Questions

1. Project Overview & Purpose

BridgeIQ is a full-stack Technical Business Analyst portfolio project. It simulates a real consulting engagement where a BA is brought in to diagnose business problems, collect and analyse data, redesign processes, define requirements, and deliver a live AI-powered solution.

It is NOT just a dashboard. It is a complete BA engagement -- every layer a hiring manager looks for:

- Business layer: BRD, stakeholder analysis, process maps, risk register, user stories, RACI matrix
- Data layer: 6-table relational schema, 16,000+ rows of synthetic data, 10 SQL queries
- Application layer: 12-page Streamlit app with real-time charts, AI integration, ML model
- AI layer: 6 Claude-powered features covering insights, feedback, requirements, and interview prep

Why Was This Built?

Technical BA roles require candidates who can bridge business analysis and technical execution. Most portfolios show only one side. BridgeIQ shows both -- it demonstrates that the analyst can write SQL, train ML models, call APIs, design data models, and produce professional BA documentation simultaneously.

Who Is the Target Audience?

- Hiring managers reviewing the GitHub repository
- Recruiters who click the live app link
- Technical interviewers looking for SQL depth and AI awareness
- Business stakeholders who want to see structured thinking and communication

What Makes It Stand Out?

- ML Churn Predictor trained on the live data -- rare in BA portfolios
- Live SQL Playground where anyone can query the database in-browser
- PDF export of executive report -- practical artefact a real stakeholder would receive
- 9-tab dashboard with sentiment analysis, anomaly detection, capacity planning, 3D scatter
- All BA artefacts (BRD, user stories, risk register) are live inside the app, not buried in PDFs
- Interview Simulator that takes any job description and returns tailored prep -- directly useful to recruiters

2. The Business Scenario -- Apex Solutions

Apex Solutions is a fictional B2B SaaS company. The scenario is designed to mirror a real BA engagement -- not a toy problem. Three critical business problems are diagnosed and solved.

Problem 1 -- 22% Annual Churn Rate

- 110 of 500 customers lost in the past year
- Approximately \$1.8M ARR at risk (industry SaaS benchmark is ~9% churn)
- Root cause: No early warning system. CSMs manually review spreadsheets every Monday.
- Detection lag: 7 days average. By the time a customer is flagged, they have usually already decided to leave.
- BridgeIQ solution: Composite Risk Score computed daily. Automated alerts when score ≥ 50 .

Problem 2 -- 18% Onboarding Failure Rate

- 90 customers stuck mid-onboarding process (never reached Go-Live stage)
- Incomplete onboarding customers are 2.4x more likely to churn within 6 months
- Root cause: Manual process, no visibility into where customers are stuck or why
- Top blockers: IT approval delays, data migration issues, training no-shows
- BridgeIQ solution: Onboarding dashboard with blocker analysis and completion tracking by plan

Problem 3 -- 28-Hour Average Ticket Resolution

- 35% of SLA breaches on Critical/High priority tickets
- CSAT declining as customers wait too long for resolution
- No sentiment analysis on ticket descriptions -- pain points are invisible
- BridgeIQ solution: SLA compliance dashboard, sentiment analysis on 4,000 ticket descriptions, anomaly detection for ticket volume spikes

The AS-IS Process (Before BridgeIQ)

Every Monday at 9AM, a CSM opens three separate spreadsheets: usage data, billing data, and support tickets. They manually cross-reference them (2-4 hours). If they spot something concerning, they email the CS Director. The director may respond 1-2 days later. If a customer is identified as at-risk, outreach happens reactively -- often after the customer has already decided to leave.

Pain Points

No integrated data. No scoring criteria. No SLA on escalation. No audit trail. Fully manual and reactive.

The TO-BE Process (With BridgeIQ)

At 6AM daily, BridgeIQ automatically pulls data from all 5 sources, computes a Composite Risk Score per customer, and routes alerts. Score < 30 = monitored. Score 30-49 = CSM reviews weekly. Score 50-79 = Slack alert within 1 hour. Score ≥ 80 = CS Director notified immediately. Intervention is logged with timestamp and owner. 88% reduction in CSM manual review time.

3. Technology Stack

Component	Technology	Why Chosen
Language	Python 3.11	Core programming language. Chosen for its data science ecosystem and Streamlit compatibility.
Web Framework	Streamlit 1.57	Converts Python scripts into interactive web apps. No frontend code needed. Pages, widgets, and charts are easy to build.
Charts	Plotly 5.x	Interactive charting library. Used for bar, area, scatter, 3D scatter, waterfall, violin, heatmap, and more.
Database	SQLite	Lightweight file-based SQL database. The entire dataset (6 tables, 16K+ rows) lives in a single file.
AI	Anthropic Claude API	Powers 6 AI features. Sonnet 4.6 for complex tasks (insights engine, requirements generator, etc.).
ML	scikit-learn	Logistic Regression model for churn prediction. StandardScaler for feature normalisation.
NLP	TextBlob	Sentiment analysis on 4,000 support ticket descriptions. Returns polarity score (-1 to +1).
PDF	fpdf2	Pure-Python PDF generation. Used in the app for the Executive Report export button, a PDF report.
Data Gen	Faker + NumPy	Generates realistic synthetic data: company names, dates, amounts, ticket descriptions, etc.
Docs	python-docx, python-pptx	Generates the BRD (.docx) and executive presentation (.pptx) programmatically.
Config	python-dotenv	Loads the ANTHROPIC_API_KEY from a local .env file without hardcoding credentials.
Stats	statsmodels	Used for OLS trendline in scatter charts (Plotly Express trendline='ols' dependency).
Hosting	Streamlit Cloud	Free hosting for Streamlit apps connected to a public GitHub repo. Auto-redeploys on every commit.

4. Project File Structure

Every file has a specific purpose. Nothing is generated randomly.

BridgeIQ/

Root project folder

README.md

GitHub landing page. Contains live demo link, feature table, tech stack, instructions.

requirements.txt

All Python dependencies. Streamlit Cloud reads this to install packages on deploy.

.gitignore

Prevents sensitive files (.env, API keys) and large binaries from being pushed to GitHub.

BridgeIQ_Complete_Guide.pdf

This document. Comprehensive explanation of the entire project.

data/

All data-related files

data/generate_data.py

Synthetic data generator. Uses Faker for realistic names/companies, NumPy for distributions. Run once to populate all CSVs and the SQLite DB.

data/customers.csv

500 customer records: ID, company, industry, plan, MRR, region, size, contract date, churn date, status, health score, NPS.

data/transactions.csv

~4,000 billing events: subscriptions, renewals, upgrades, downgrades, refunds. Includes negative amounts for refunds.

data/support_tickets.csv

4,000 tickets with descriptions, categories, priorities, resolution times, and CSAT scores. Descriptions are realistic (used for sentiment analysis).

data/product_usage.csv

8,000 usage sessions: customer, date, feature used, session minutes, actions count.

data/onboarding.csv

500 onboarding records (one per customer): stage reached, completion date, days to complete, blocker reason.

data/employees.csv

120 internal employees: department, role, salary, location. Used in Capacity Planning tab.

data/bridgeiq.db

SQLite database containing all 6 tables. The app reads exclusively from this file.

sql/

10 standalone SQL query files

sql/01_kpi_overview.sql

High-level KPIs: active customers, MRR, churn rate, health score averages.

sql/02_churn_analysis.sql

Churn breakdown by industry, plan, region. Identifies highest-risk segments.

sql/03_revenue_trends.sql

Monthly net and gross revenue trends. Includes upgrade/downgrade impact.

sql/04_support_ticket_analysis.sql

SLA compliance by priority, resolution time distribution, category breakdown.

sql/05_product_usage_analysis.sql

Feature adoption rates, session duration, most/least used features.

sql/06_onboarding_analysis.sql

Completion rates, days to complete by plan, blocker frequency.

`sql/07_at_risk_customers.sql`

Composite Risk Score CTE: health + tickets + usage + onboarding + NPS.

`sql/08_cohort_retention.sql`

Signup quarter cohorts, retention curves, MRR retention by cohort.

`sql/09_employee_workforce.sql`

Headcount by department, salary distribution, hire trends.

`sql/10_executive_summary_view.sql`

Single query that returns all key metrics for the executive dashboard.

app/

Streamlit application

`app/app.py`

The entire app: ~3,200 lines. All 12 pages, all CSS, all charts, all AI calls, all SQL, all navigation.

`app/.env`

Your ANTHROPIC_API_KEY. This file is gitignored and never pushed to GitHub.

`app/.env.example`

Template showing the required environment variable name. Safe to commit.

docs/

BA documents

`docs/build_brd.py`

Script that generates the Word .docx BRD programmatically using python-docx.

`docs/BridgeIQ_BRD.docx`

11-section Business Requirements Document: problem statement, objectives, scope, stakeholders, requirements, risks, success metrics.

`docs/data_dictionary.csv`

48 columns documented: table, column name, data type, nullable, PK/FK, description, example value, business rule.

`docs/user_stories.csv`

15 Jira-importable user stories with persona, feature, benefit, acceptance criteria, story points, MoSCoW priority, epic.

`docs/generate_guide.py`

This script. Generates BridgeIQ_Complete_Guide.pdf.

presentation/

Executive presentation

`presentation/build_deck.py`

Script that generates the 10-slide PowerPoint using python-pptx.

`presentation/BridgeIQ_Executive_Presentation.pptx`

10-slide deck: problem, data model, KPIs, process redesign, solution architecture, roadmap, ROI.

diagrams/

Process and data diagrams

`diagrams/erd.md`

Entity Relationship Diagram in Mermaid syntax. Shows all 6 tables and their foreign key relationships.

`diagrams/process_asis.md`

AS-IS churn detection process in Mermaid flowchart syntax. 8 steps, 5 pain points.

`diagrams/process_tobe.md`

TO-BE automated churn prevention workflow in Mermaid. Shows BridgeIQ's 4-tier alert system.

5. Data Layer -- 6 Tables & Schema

All data is synthetic, generated by `data/generate_data.py` using Faker and NumPy. The schema is designed to mirror a real B2B SaaS company's data model. The 6 tables are related via foreign keys and together support every chart, query, and AI feature in the app.

Table: customers (500 rows)

Central table. Every other table references this via `customer_id`. Contains all account-level data.

Column	Type	Description
<code>customer_id</code>	TEXT PK	CUST0001-CUST0500. Unique identifier.
<code>company_name</code>	TEXT	Realistic B2B company names from Faker.
<code>industry</code>	TEXT	One of 10 industries: Healthcare, Finance, Retail, etc.
<code>plan_type</code>	TEXT	Starter / Growth / Enterprise. Determines MRR range.
<code>mrr</code>	REAL	Monthly Recurring Revenue in USD. Starter: \$500-\$999. Growth: \$1000-\$4999. Enterprise: \$5000+.
<code>region</code>	TEXT	One of 5 regions: North America, Europe, Asia Pacific, Latin America, Middle East.
<code>health_score</code>	REAL	0-100. Computed daily from usage, ticket, and billing signals.
<code>nps_score</code>	INTEGER	0-10. Net Promoter Score from most recent survey.
<code>status</code>	TEXT	Active or Churned. Determines if <code>churn_date</code> is populated.
<code>churn_date</code>	DATE	NULL for Active customers. Set when <code>status</code> = Churned.

Table: transactions (~4,000 rows)

One row per billing event. Supports revenue trend charts and MRR analysis.

Column	Type	Description
<code>transaction_id</code>	TEXT PK	TXN00001-TXN04000. Unique identifier.
<code>customer_id</code>	TEXT FK	Links to <code>customers.customer_id</code> .
<code>amount</code>	REAL	Positive for charges, negative for refunds/downgrades.
<code>transaction_type</code>	TEXT	Subscription / Renewal / Upgrade / Downgrade / One-time Add-on / Refund.
<code>status</code>	TEXT	Completed / Pending / Failed.

Table: support_tickets (4,000 rows)

One row per support ticket. Used for SLA analysis, category breakdown, and sentiment analysis.

Column	Type	Description
<code>ticket_id</code>	TEXT PK	TKT00001-TKT04000.
<code>customer_id</code>	TEXT FK	Links to <code>customers</code> .
<code>category</code>	TEXT	Technical Issue, Billing, Feature Request, etc.
<code>priority</code>	TEXT	Critical / High / Medium / Low. Determines SLA target.
<code>resolution_time_hours</code>	REAL	Hours from creation to resolution. SLA: Critical<=8h, High<=24h, Medium<=48h, Low<=96h.
<code>description</code>	TEXT	Free-text issue description. Used for TextBlob sentiment analysis.

Table: product_usage (8,000 rows)

One row per usage session. Used to identify ghost accounts and active users.

Column	Type	Description
usage_id	TEXT PK	USG00001-USG08000.
customer_id	TEXT FK	Links to customers.
feature	TEXT	One of 10 features: Dashboard, Reporting, API, Integrations, etc.
session_minutes	REAL	Duration capped at 120 minutes.
actions_count	INTEGER	Clicks, form submissions, exports counted as actions.

Table: onboarding (500 rows)

One-to-one with customers. Tracks the onboarding journey of each account.

Column	Type	Description
onboarding_id	TEXT PK	ONB0001-ONB0500.
customer_id	TEXT FK	One record per customer.
stage	TEXT	Account Setup / Data Migration / User Training / Integration Config / Go-Live.
completed	INTEGER	1 = reached Go-Live. 0 = still in progress or stuck.
days_to_complete	INTEGER	NULL if not completed. Used in capacity and health analysis.
blocker	TEXT	Primary reason for being stuck: IT approval, data migration, training no-show, etc.

Table: employees (120 rows)

Internal Apex Solutions staff. Used in Capacity Planning tab.

Column	Type	Description
employee_id	TEXT PK	EMP0001-EMP0120.
department	TEXT	Customer Success, Support, Engineering, Sales, etc.
role	TEXT	Job title within department.
salary	REAL	Annual base salary in USD.
location	TEXT	One of 7 locations including Remote.

6. SQL Analytics Layer -- 10 Queries

Each SQL file in the `sql/` folder is a standalone query demonstrating a specific analytics capability. These are the same queries (adapted with Python f-strings for filtering) that power the Executive Dashboard charts.

01 -- KPI Overview

Aggregate metrics across all customers. Uses conditional aggregation (`COUNT CASE WHEN`) to compute active vs churned counts, MRR sums, and churn rates in a single pass.

SQL Techniques

Demonstrates: conditional aggregation, `ROUND()`, single-row summary queries.

02 -- Churn Analysis

Breaks churn down by industry, plan type, and region. Shows which segments have the highest churn rates and absolute customer losses.

SQL Techniques

Demonstrates: `GROUP BY` multi-dimension, percentage calculations, sorting by derived columns.

03 -- Revenue Trends

Monthly net and gross revenue using `strftime()` for date grouping. Joins transactions and customers to allow plan/industry filtering.

SQL Techniques

Demonstrates: date functions, `JOIN`, `SUM` with `CASE` for conditional aggregation, time-series output.

04 -- Support Ticket Analysis

SLA compliance by priority using `CASE WHEN` thresholds per priority level. Shows breach counts and breach percentages. Also provides category breakdown with average resolution times.

SQL Techniques

Demonstrates: multi-condition `CASE WHEN`, `GROUP BY`, `ORDER BY` with `CASE` for custom sort order.

05 -- Product Usage Analysis

Feature adoption ranking: sessions, average duration, total actions per feature. Identifies which features are most and least used across all customers.

SQL Techniques

Demonstrates: `GROUP BY` on categorical data, `AVG`, `SUM`, multi-metric aggregation.

06 -- Onboarding Analysis

Completion rates by plan, average days to complete, blocker frequency. Joins onboarding with customers to enable plan-level segmentation.

SQL Techniques

Demonstrates: `JOIN`, conditional `COUNT`, `AVG` with `CASE` for conditional averaging (only completed records).

07 -- At-Risk Customers

The Composite Risk Score query. Uses a CTE to pre-aggregate ticket counts and usage sessions, then applies a scoring formula using nested CASE WHEN expressions across 5 risk signals.

SQL Techniques

Demonstrates: Common Table Expressions (CTEs), multi-table LEFT JOINs, nested CASE WHEN scoring, COALESCE for NULL handling, ORDER BY risk score.

08 -- Cohort Retention

Groups customers by signup quarter (strftime('%Y-Q', contract_start)) and tracks how many are still active. Shows retention curves over time for each cohort.

SQL Techniques

Demonstrates: date-based cohort grouping, strftime, retention calculations, cohort analysis pattern.

09 -- Employee Workforce

Headcount by department, salary ranges, role distribution. Powers the Capacity Planning tab to show current CSM and support team sizes.

SQL Techniques

Demonstrates: GROUP BY on HR data, COUNT, salary aggregations.

10 -- Executive Summary View

A single comprehensive query that joins all key metrics into one result set. Designed to power the top-level KPI strip with minimal database round trips.

SQL Techniques

Demonstrates: subquery aggregation, multi-table summary, efficient single-pass analytics.

The Composite Risk Score Formula

The most important SQL construct in the project. This formula powers both the at-risk customer table and the Churn Predictor page:

Risk Score Components

Health Score < 60: +30 pts | Health 60-75: +15 pts | Health >= 75: 0 pts
Ticket Count > 15: +20 pts | Tickets 8-15: +10 pts | Tickets < 8: 0 pts
Usage Sessions < 3: +25 pts | Sessions 3-8: +12 pts | Sessions > 8: 0 pts
Onboarding Incomplete: +15 pts | Completed: 0 pts
NPS Score < 5: +10 pts | NPS >= 5: 0 pts
Total possible: 100 pts. Score >= 50 = HIGH RISK. Score >= 80 = CRITICAL.

7. App Architecture -- How Streamlit Works

Streamlit's execution model is simple but important to understand. Every time a user interacts with any widget (clicks a button, selects a dropdown, moves a slider), the entire Python script re-runs from top to bottom. The output (charts, text, widgets) is re-rendered fresh each time.

Execution Order in app.py

1. Imports -- all libraries loaded
2. Page config -- `st.set_page_config()` sets title, icon, layout. Must be FIRST Streamlit call.
3. CSS injection -- `st.markdown(css, unsafe_allow_html=True)` injects global styles
4. Constants -- colour palette, DB path, API key loaded from environment
5. Helper functions -- `chart_layout()`, `query()` defined once, reused everywhere
6. Navigation -- `selectbox` determines which page the user is on (the `page` variable)
7. Page routing -- `if/elif` chain: `if page == 'Home': ... elif page == 'Dashboard': ...`
8. Page content -- whichever branch matches runs all its SQL, charts, and widgets

Session State

`st.session_state` is a dictionary that persists between reruns within the same browser session. BridgeIQ uses it for: `go_to_page` (programmatic navigation target), `deep_link_consumed` (flag to read query params only once on load).

Caching

`@st.cache_data(ttl=300)` is applied to the `query()` function. This means the same SQL query is only executed against SQLite once every 5 minutes. Subsequent reruns return the cached DataFrame. The ML model and sentiment analysis use `ttl=3600` (1 hour cache) because they are compute-intensive.

Navigation Pattern

The navigation uses a `st.selectbox()` at the top of the page. The selected value (`page`) determines which `elif` branch executes. Programmatic navigation (from the Home page feature card buttons) works by setting `st.session_state['go_to_page'] = target`, then calling `st.rerun()`. On the next run, the `go_to_page` value forces the `selectbox` index before rendering, then is consumed (set to `None`) so it doesn't persist.

Database Connection

SQLite is accessed via Python's built-in `sqlite3` module. The `query()` helper opens a connection, runs `pd.read_sql_query(sql, conn)`, closes the connection, and returns a DataFrame. The DB file path is relative to `app.py` using `os.path.dirname(__file__)` to work both locally and on Streamlit Cloud.

Section 8 -- [HOME] Home

[HOME] Home

Landing page. First thing any visitor sees.

Overview

The Home page solves a real UX problem: if you land directly on the Executive Dashboard, you see data without context. A recruiter or hiring manager would not understand what they are looking at. The Home page answers: What is this? What problem does it solve? What can I do here?

Feature Breakdown

Hero Section

Large animated gradient title. Badge strip showing 16,000+ rows, 10 SQL queries, 6 AI features, 15 user stories. This immediately signals the depth of the project.

The Scenario

Explains Apex Solutions: who they are, what problems they face, why a BA was brought in. Sets the stage for every other page.

Three Problems

Each problem has its data signal, business impact, and a metric card. Shows BA structured thinking: quantify first, then diagnose.

Feature Cards

12 clickable cards, one per app page. Each card shows the page icon, title, description, and an 'Open' button. Clicking navigates directly to that page via `st.session_state`.

Section 9 -- [DASHBOARD] Executive Dashboard

[DASHBOARD] Executive Dashboard

9-tab BI dashboard. The centrepiece of the app.

Overview

This is the most complex page. It has 9 tabs, each showing a different analytics perspective. All charts respond to 3 inline filters at the top: Plan Type, Region, and Industry. The filters use Streamlit's `st.expander` and `st.selectbox`. The filter values are fed into SQL WHERE clauses via the `_wc()` and `_ac()` helper functions.

Feature Breakdown

Inline Filters

Plan Type, Region, Industry dropdowns inside an expander. `_wc()` builds the WHERE clause. `_ac()` builds the AND clause (for queries that already have a WHERE).

Hero KPI Strip

6 metrics: Total ARR, MRR at Risk, At-Risk Accounts, Open Tickets, Avg Health Score, Churn Rate. All computed live from the filtered dataset. Pure HTML div with flex layout.

Mini KPI Cards

6 cards below the strip: Active Customers, Total MRR, Total ARR, Churn Rate, Avg Health, Onboarding %. Each has a progress bar.

Tab 1 Overview

Monthly revenue area chart (gross vs net), health score gauge, customer mix treemap, at-risk table with colour-coded risk scores, churn by segment table.

Tab 2 Revenue

Revenue waterfall by transaction type, MRR by plan pie, payment method distribution, revenue by industry, month-over-month growth.

Tab 3 Customer Health

Health score histogram, health violin by plan, churn heatmap by plan x region, usage vs health scatter with OLS trendline, 3D segmentation scatter.

Tab 4 Support

Open ticket count, SLA breach bar by priority, category breakdown, resolution time box plot, ticket sentiment analysis (3 charts).

Tab 5 Onboarding

Completion gauge, blocker pie, completion by plan bar, onboarding impact on churn comparison.

Tab 6 Cohort Retention

Cohort retention table, retention curve by quarter, MRR by cohort.

Tab 7 SaaS Benchmarks

BridgeIQ metrics vs industry benchmarks from Baremetrics, Gainsight, Zendesk, Totango.

Tab 8 Anomaly Detector

Statistical anomaly detection using mean + 1 standard deviation threshold. Identifies churn spikes by industry, ticket volume surges by month, ghost accounts (active customers with <3 usage sessions), SLA breach concentration. One-click AI root cause analysis via Claude Haiku.

Tab 9 Capacity Planning

Current CSM headcount vs customer count, customers-per-CSM ratio vs benchmark, 12-month growth projection, CSM hire gap forecast, department headcount bar.

PDF Export

Build PDF Report button in Tab 1. Uses fpdf2 to generate a styled PDF with KPIs, business summary, and recommendations. Downloads instantly via st.download_button.

Section 10 -- [BRAIN] AI Insights Engine

[BRAIN] AI Insights Engine

Claude reads live data and writes a McKinsey-style executive brief.

Overview

This page pulls the current KPIs, revenue trends, support metrics, and onboarding data from the database, formats them into a structured data context, and sends them to Claude Sonnet 4.6. The model generates a multi-section executive intelligence brief with business situation, critical signals, risks, and recommended actions.

Feature Breakdown

Data Context

The prompt includes: total ARR, MRR, churn rate, health score, at-risk count, open tickets, onboarding rate, top 3 at-risk customers, churn by industry table, SLA breach summary.

Prompt Engineering

Claude is instructed to respond as a McKinsey consultant. Specific output sections are requested: Executive Situation, Critical Signals (top 3), Revenue at Risk analysis, Recommended Actions (prioritised), and a Board-Ready Summary.

Model

claude-sonnet-4-6. The most capable model, chosen because this is a complex synthesis task requiring business judgment, not just extraction.

Output

Markdown rendered with `st.markdown()`. Includes headers, bullet points, and bold key metrics.

Section 11 -- [ROBOT] AI Feedback Analyzer

[ROBOT] AI Feedback Analyzer

Paste raw customer feedback. Get structured BA analysis instantly.

Overview

This page addresses a real BA workflow: you have unstructured customer feedback (survey responses, emails, interview notes) and need to turn it into structured requirements. The analyst pastes the text and Claude returns pain points, severity ratings, affected personas, and draft user stories.

Feature Breakdown

Input

st.text_area for pasting raw feedback text.

Output sections

Pain points (numbered, with severity High/Medium/Low), Affected user personas, Root cause themes, Draft user stories in As a / I want / So that format, Recommended BA next steps.

Model

claude-haiku-4-5. Faster and cheaper than Sonnet. Sufficient for structured extraction tasks where the format is clearly specified.

Why useful

Demonstrates that the analyst understands how to use AI as a BA productivity tool, not just as a chatbot.

Section 12 -- [CLIPBOARD] AI Requirements Generator

[CLIPBOARD] AI Requirements Generator

Describe a business problem. Get a full BA deliverables package.

Overview

This page takes a problem statement and stakeholder information and generates a complete set of BA artefacts. It is the most comprehensive AI feature because it produces multiple output types in a single call.

Feature Breakdown

Inputs

Problem description text area, primary stakeholder dropdown (CEO, VP CS, Product Manager, etc.), priority level (High/Medium/Low), output type multiselect (BRD Excerpt, User Stories, Acceptance Criteria, UAT Test Cases, Risk Register).

BRD Excerpt

Problem Statement, Business Objectives (3-4), Scope (In/Out), Assumptions, Constraints, Success Metrics table.

User Stories

5-7 stories with persona, feature, benefit, story points, MoSCoW priority.

Acceptance Criteria

Given/When/Then format, 2-4 criteria per story.

UAT Test Cases

Test ID, Scenario, Steps, Expected Result, Pass/Fail Criteria.

Risk Register

5 risks with Probability, Impact, and Mitigation Strategy.

Example quick-loads

3 pre-built problem statements available so users can see the feature without typing.

Section 13 -- [TARGET] Interview Simulator

[TARGET] Interview Simulator

Paste a job description. Get a tailored interview prep package.

Overview

This page directly serves the job search use case. A recruiter or hiring manager visits the live app. The analyst shows them this page with a real JD pasted in. Claude generates content that is specific to that role -- not generic BA interview advice.

Feature Breakdown

Input

Job description text area. Focus area dropdown (Stakeholder Management, SQL/Data, AI/ML, Process Design, Agile/Scrum, System Integration). Report For dropdown (CEO, VP CS, PM, Operations Manager).

Output

Role analysis (what this role actually cares about), tailored talking points connecting BridgeIQ work to the JD requirements, 8 likely interview questions with model answers, 60-second elevator pitch, red flags to avoid.

Why powerful

The analyst can paste any BA job description and immediately show the interviewer exactly how BridgeIQ maps to their requirements. This turns the app into a live interview tool.

Section 14 -- [SEARCH] Customer 360

[SEARCH] Customer 360

Full account intelligence for one customer in a single view.

Overview

This page answers: what does a CSM need to know about a customer before any interaction? It pulls data from all 5 tables for a single customer and displays it in one comprehensive view.

Feature Breakdown

Customer selector

st.selectbox with all 500 customers. Searchable by company name.

KPI strip

Company, MRR, ARR, Health Score, NPS, Risk Score (computed live from the formula), Status.

Risk Factor Breakdown

Horizontal bar chart showing how many points each signal contributed to the risk score. CSMs can see exactly WHY a customer is high risk.

Product Usage by Feature

Bar chart of feature usage sessions, coloured by count. Shows which features the customer uses and which they ignore.

Transaction History

Last 20 billing events in a styled table.

Support Ticket History

Last 15 tickets with priority colour-coding.

Journey Timeline

Plotly Gantt/timeline chart showing the customer's full lifecycle: contract start, onboarding period, all transactions, and recent support tickets plotted on a time axis.

Onboarding Status

Colour-coded block showing stage, start date, days to complete, and blocker (if any).

AI Account Summary

Button triggers Claude Haiku to generate a CSM-style account brief: situation, top 3 risks, recommended actions, talking points for the next call.

Section 15 -- [RULER] BA Artifacts

[RULER] BA Artifacts

All BA deliverables live inside the app -- not buried in PDFs.

Overview

This page demonstrates that the analyst produced real BA documentation, not just code. Every artefact is rendered interactively inside the app.

Feature Breakdown

User Stories tab

15 sprint-ready stories in a filterable table. Filter by Priority, Epic, Status. Each story has: ID, Epic, Persona, Feature, Benefit, Acceptance Criteria, Story Points, MoSCoW priority.

Risk Register tab

6 identified risks with: ID, Category, Description, Probability, Impact, Risk Score, Status, Mitigation Strategy, Owner.

RACI Matrix tab

Responsibility matrix across 5 stakeholder groups for 8 process activities.

Process Maps tab

AS-IS and TO-BE process flows rendered as Mermaid diagrams inside the app. Shows the before/after transformation.

Traceability Matrix tab

Links each user story to a business objective, data source, and success metric. Shows requirements coverage.

Section 16 -- [BULB] What-If Revenue Simulator

[BULB] What-If Revenue Simulator

Move sliders. See live ARR impact.

Overview

This page lets stakeholders model the financial impact of BridgeIQ interventions without touching a spreadsheet. It is the ROI calculator for the engagement.

Feature Breakdown

Sliders

Current Churn % (default: live data value), Target Churn % (what could be achieved), Onboarding Completion % (current vs target), Avg Resolution Hours (current vs target).

Calculations

For each slider, the ARR impact is computed: $\text{churn recovery} = (\text{current_churn} - \text{target_churn}) * \text{MRR} * 12 * 0.35$ (35% early intervention success rate). Similar logic for onboarding and resolution time.

Waterfall Chart

Plotly go.Waterfall shows the base ARR, plus each intervention's contribution, to the projected total ARR. Visual and immediately understandable by non-technical stakeholders.

Impact Cards

Current ARR, MRR at Risk, Recoverable with BridgeIQ, Annual ROI if deployed for 12 months.

Section 17 -- [CRYSTAL] Churn Predictor

[CRYSTAL] Churn Predictor

ML model trained on live data. Per-customer churn probability.

Overview

This is the most technically impressive page for a BA portfolio. It shows the analyst can take raw business data, train a classification model, evaluate it, interpret it, and present it in a business context.

Feature Breakdown

Data Preparation

A JOIN query pulls 7 features per customer: MRR, health score, NPS score, usage sessions, ticket count, onboarding completed flag, days to complete. The target variable is churned (1) or active (0).

Model

LogisticRegression from scikit-learn. Max iterations: 1000. StandardScaler normalises all features before training (important because MRR is in the thousands while NPS is 0-10).

Train/Test Split

80% train, 20% test, stratified by churn label to preserve class balance.

Metrics

Accuracy (% of predictions correct) and ROC-AUC (ability to distinguish churned vs active regardless of threshold). Both displayed as KPI cards.

Feature Importance

Horizontal bar chart of absolute coefficient values from the logistic regression. Shows which signals most strongly predict churn.

Probability Distribution

Histogram of predicted churn probabilities for all 500 customers, coloured by whether they actually churned. Validates the model's discrimination ability.

Risk Table

Filterable table: show customers with churn probability $\geq$ X%. Sortable. Colour-coded probability column. Business-ready output.

Caching

@st.cache_data(ttl=3600) on the training function. The model is only retrained once per hour, not on every page interaction.

Section 18 -- [LAPTOP] SQL Playground

[LAPTOP] SQL Playground

Live SQL editor against the full Apex Solutions database.

Overview

This page serves two audiences: technical reviewers who want to verify the data model and query it themselves, and the analyst who wants to demonstrate SQL fluency during a screen share.

Feature Breakdown

Schema Reference

Collapsible expanders for each of the 6 tables, listing every column name. Always visible on the left side.

Query Editor

st.text_area pre-loaded with a default query. Dropdown to load 5 example queries: top customers by MRR, churn by industry, SLA breach, feature usage, onboarding blockers.

Query Execution

On Run Query button click, the SQL is checked for forbidden keywords (DROP, DELETE, INSERT, UPDATE, ALTER, CREATE, TRUNCATE). Read-only mode -- SELECT only. Results displayed in a dataframe.

CSV Download

st.download_button appears after a successful query. Downloads the result as a CSV file.

Why this matters

Most BA portfolios have static screenshots of SQL results. This lets anyone run any query and see the real data. It is a live proof of the SQL layer's depth.

Section 19 -- [USER] Engagement Summary

[USER] Engagement Summary

Profile, project stats, skills, ROI calculator, and CTA.

Overview

This page serves as a live resume and contact card for the analyst. It is optimised for a recruiter or hiring manager who has spent a few minutes in the app and wants to know who built it.

Feature Breakdown

Stats Strip

5 metrics: Years BA Experience, Data Rows Generated, SQL Queries Written, AI Features Built, User Stories Delivered.

Skills Grid

3-column grid of skill badges across: BA Skills, Data & SQL, AI & Python.

ROI Calculator

Shows the financial case for hiring a Technical BA: current ARR, MRR at risk from churn, recoverable revenue with BridgeIQ, and annual ROI if deployed for 12 months.

Project Timeline

Week-by-week build log: Week 1 Data Engineering, Week 2 SQL, Week 3 Streamlit, Week 4 AI Integration, Week 5 BA Deliverables, Shipped.

CTA Section

Email button (mailto link), GitHub button, open-to-BA-roles badge, IEEE-Published badge, contact details.

20. AI Integration -- How Claude API Works

BridgeIQ uses the Anthropic Claude API for 6 AI-powered features. The API is called via the official anthropic Python SDK.

Authentication

The API key is stored in `app/.env` as `ANTHROPIC_API_KEY=sk-ant-...`. The `dotenv` library loads it: `API_KEY = os.getenv('ANTHROPIC_API_KEY', '')`. On Streamlit Cloud, the key is set as a Secret in the app settings (not in `.env`). The app checks if `API_KEY` is empty before making calls and shows a friendly error if not configured.

API Call Pattern

Standard Call Structure

```
client = anthropic.Anthropic(api_key=API_KEY)
msg = client.messages.create(
    model='claude-sonnet-4-6', # or claude-haiku-4-5-20251001
    max_tokens=2000,
    messages=[{'role': 'user', 'content': prompt}]
)
result = msg.content[0].text
```

Model Choice Logic

- Claude Sonnet 4.6 (claude-sonnet-4-6): Used for AI Insights Engine and Requirements Generator. These tasks require genuine synthesis and business judgment across large contexts.
- Claude Haiku 4.5 (claude-haiku-4-5-20251001): Used for Feedback Analyzer, Interview Simulator, Account Summary, and Anomaly Root Cause. These tasks are structured extraction/generation where speed matters more than depth.
- Cost rationale: Haiku is approximately 15x cheaper than Sonnet. Using Haiku for high-frequency tasks significantly reduces API costs in a portfolio app with unpredictable traffic.

Prompt Engineering Principles Used

- Role assignment: 'You are a McKinsey consultant...' or 'You are a CSM reviewing an account...' This frames the response style and vocabulary.
- Data injection: Live database values are interpolated into the prompt as structured text blocks. Claude always works with current data, not static examples.
- Output structure: Required sections are explicitly listed with markdown headers (`## SECTION NAME`). This enforces consistent, parseable output.
- Format constraints: Responses are rendered as Markdown via `st.markdown()`. Bold, headers, and bullet points all render correctly.
- Error handling: All API calls are wrapped in `try/except`. If the API fails, `st.error()` shows a friendly message instead of crashing the app.

21. Navigation & Deep Links

Primary Navigation -- Selectbox

The primary navigation is a `st.selectbox` at the top of the page containing all 12 page names. The selected value (page variable) drives an `if/elif` chain that renders the correct page content. Label visibility is set to collapsed so no label text appears above the dropdown.

Programmatic Navigation -- Home Page Buttons

The Home page feature cards have 'Open' buttons. When clicked, they set `st.session_state['go_to_page'] = target_page` and call `st.rerun()`. On the next run, before the selectbox renders, the code checks if `go_to_page` is set, finds its index in the `PAGES` list, sets `_nav_index` to that index, and clears `go_to_page`. The selectbox renders with the correct index, selecting the target page automatically.

Deep Links -- ?page= Query Parameter

The app reads `?page=` from the URL on the very first load of a session (detected via `deep_link_consumed` in session state). This allows sharing links like `bridgeiq.streamlit.app?page=Churn+Predictor` that open directly to a specific page. After the first load, query params are never re-read during reruns -- this prevents the parameter from overriding subsequent user navigation (a bug that caused the double-click issue before this fix).

22. CSS & Visual Design System

All custom CSS is injected via `st.markdown(css_string, unsafe_allow_html=True)` at the top of the script. It applies globally to all pages.

Colour Palette

Name	Hex Code	Usage
Background	#0f1117	App base background
Card BG	#0d1b2a	KPI cards, containers
Border	#1e3a5f	All borders and dividers
Primary Blue	#4F8EF7	Headings, accents, active states
Success Green	#22c55e	Positive metrics, good health
Warning Yellow	#f59e0b	Medium risk, caution states
Danger Red	#ef4444	High risk, churn, SLA breach
Text Light	#f1f5f9	Primary text on dark background
Text Muted	#64748b	Labels, captions, secondary text

CSS Animation Classes

pulse-dot:	The green live indicator dot. Scales box-shadow from 0 to 7px and back every 2 seconds.
slide-up:	Applied to KPI cards. Cards fade in from 18px below their final position over 0.5 seconds.
gradient-x:	Applied to page titles. Cycles the background-position of a 4-colour gradient left-to-right continuously.
danger-flash:	Applied to red alert boxes. Pulses a red box-shadow to draw attention.
border-glow:	Applied to the hero KPI strip. Pulses a blue outer glow and subtle inner glow.
float:	Applied to AI page icons. Gently bobs up and down 6px every 3 seconds.
count-in:	Applied to KPI values. Scales from 80% to 100% size on first render.

Key Component Styles

- `.kpi-card`: Dark gradient card with left blue border, hover lift effect (`translateY -4px`), animated slide-up on load.
- `.hero-strip`: Full-width flex container with border-glow animation. Holds the 6 top-line KPIs.
- `.section-header`: Section title with left blue accent border and bottom rule. Consistent across all pages.
- `.page-title`: 36px animated gradient text for page headings.
- `.badge`: Inline pill badge in 5 colour variants (blue, green, red, yellow, purple).
- `[data-baseweb='select']`: Overrides Streamlit's default selectbox styling to match the dark theme.
- `.stTabs`: Overrides tab styling -- dark background, blue active indicator.
- `[data-testid='stSidebar']`: Hidden entirely with `display:none !important` (sidebar removed, all navigation is inline).

23. BA Deliverables Explained

Every deliverable was produced as part of a structured BA engagement, not as an afterthought. Each one maps to a standard BA responsibility.

Business Requirements Document (BRD)

11 sections covering: Executive Summary, Problem Statement, Business Objectives (SMART), Project Scope (in/out), Stakeholder Analysis, Functional Requirements, Non-Functional Requirements, Assumptions & Constraints, Risks, Success Metrics, Approval signatures. Generated programmatically by docs/build_brd.py using python-docx.

15 User Stories

Sprint-ready stories in the standard format. Each has: Story ID, Epic (Customer Health / Onboarding / Support / Analytics / AI Features), Persona, Feature, Benefit, Detailed Acceptance Criteria, Story Points (Fibonacci), MoSCoW priority (Must Have / Should Have / Could Have), Status, Implementation Notes. Stored in docs/user_stories.csv (Jira-importable format).

Risk Register

6 identified risks: Data Quality Risk, AI Hallucination Risk, Scope Creep Risk, Adoption Risk, API Dependency Risk, Data Privacy Risk. Each has: risk description, probability (1-5), impact (1-5), risk score (P x I), status (Open/Mitigated), mitigation strategy, and risk owner.

RACI Matrix

Responsibility matrix across 5 stakeholders (CS Director, CSM, IT Director, Product Manager, Data Analyst) for 8 process activities. R=Responsible, A=Accountable, C=Consulted, I=Informed.

AS-IS / TO-BE Process Maps

Both rendered as Mermaid flowcharts. AS-IS shows the 8-step manual process with pain points highlighted. TO-BE shows BridgeIQ's 4-tier automated alert system with risk thresholds, intervention routing, and escalation paths.

Requirements Traceability Matrix

Links each user story to: the business objective it addresses, the data source(s) required, the success metric that validates it, and the UAT test case that verifies it. Ensures full requirements coverage and prevents scope gaps.

Data Dictionary

48 columns documented across 6 tables. For each column: table name, column name, data type, nullable, primary key flag, foreign key reference, business description, example value, and business rule/constraint.

Executive Presentation

10-slide PowerPoint deck generated by presentation/build_deck.py using python-pptx. Covers: problem statement, data model overview, key metrics, AS-IS pain points, TO-BE solution, AI feature roadmap, ROI model, next steps.

24. How to Run Locally

Clone the repository

```
git clone https://github.com/gtrhemanth/BridgeIQ-.git
cd BridgeIQ-
```

Install dependencies

```
pip install -r requirements.txt
(Creates a virtual environment first: python -m venv venv && venv\Scripts\activate)
```

Generate the data

```
cd data
python generate_data.py
cd ..
(Creates all 6 CSV files and bridgeiq.db. Run once.)
```

Add your API key

Copy app/.env.example to app/.env
Edit app/.env: ANTHROPIC_API_KEY=sk-ant-your-key-here
(Get a key from console.anthropic.com)

Run the app

```
cd app
streamlit run app.py
(Opens at http://localhost:8501)
```

Optional: Regenerate docs

```
cd docs && python build_brd.py
cd ../presentation && python build_deck.py
cd ../docs && python generate_guide.py
```

25. How to Deploy on Streamlit Cloud

Push to GitHub

```
git add .
git commit -m 'initial commit'
git push origin main
```

Go to Streamlit Cloud

Visit share.streamlit.io. Sign in with your GitHub account.

Create new app

Click 'New app'. Select repository: gtrhemanth/BridgeIQ-. Branch: main. Main file path: app/app.py.

Add the API key

In Advanced Settings, add: ANTHROPIC_API_KEY = sk-ant-your-key-here. This is Streamlit's Secret management -- the value is encrypted and never appears in logs.

Deploy

Click Deploy. Streamlit installs requirements.txt, runs app.py, and gives you a public URL. Any future git push to main auto-redeploys.

26. Key Design Decisions & Why

Single-file app (app.py)

All 12 pages are in one file (~3,200 lines) instead of using Streamlit's multipage feature (separate files per page). Reason: simplifies deployment, avoids shared state issues between pages, and allows global CSS and helper functions to be defined once.

SQLite instead of PostgreSQL

SQLite is a single file -- zero configuration, works offline, deploys to Streamlit Cloud without a database server. For a portfolio with synthetic data, PostgreSQL would add complexity with no benefit.

Synthetic data instead of real data

Using Faker and NumPy allows generating data with controlled patterns (realistic churn rates, SLA distributions, health score ranges) that demonstrate specific analytics capabilities. Real data would require anonymisation and might not show the right distribution of edge cases.

Single selectbox navigation instead of sidebar

Streamlit's sidebar navigation requires each page to be a separate file in a `pages/` folder. Single-file navigation gives full control over layout, styling, and programmatic navigation (the `go_to_page` pattern).

TextBlob instead of transformers

Transformers (HuggingFace) would give more accurate sentiment but requires downloading 400MB+ models, which is slow on Streamlit Cloud cold starts. TextBlob is 2MB and instantaneous. For demonstrating the concept of sentiment analysis in a portfolio, accuracy difference is acceptable.

Logistic Regression instead of XGBoost/Random Forest

Logistic Regression is explainable: coefficients directly give feature importance. XGBoost would be more accurate but harder to explain to a business audience. For a BA portfolio, explainability matters more than marginal accuracy improvement.

fpdf2 instead of reportlab

fpdf2 is pure Python with no system dependencies. Reportlab requires system libraries that sometimes cause issues on Streamlit Cloud. fpdf2 is simpler to install and sufficient for the structured report format needed.

@st.cache_data for all queries

Without caching, every widget interaction reruns every SQL query. With caching (`ttl=300` = 5 minutes), the same query is only executed once per 5-minute window. This makes the app feel instant even on Streamlit Cloud's free tier infrastructure.

27. Frequently Asked Questions

Q: Is the data real?

A: No. All 16,000+ rows are synthetically generated using Faker (for names and text) and NumPy (for numerical distributions). The patterns are designed to be realistic -- churn rates, health score distributions, ticket volumes -- but no real company or customer data is used.

Q: Does the AI cost money to use?

A: Yes. The Anthropic Claude API charges per token. Haiku is approximately \$0.00025 per 1K input tokens. Sonnet is approximately \$0.003 per 1K input tokens. For a portfolio app with occasional use, monthly costs are typically under \$2-5. The API key in the .env file controls all costs.

Q: Why does the churn predictor show different accuracy each time?

A: It shouldn't -- the model is cached with `@st.cache_data(ttl=3600)`. Within one hour, the model is only trained once. After the cache expires, it retrains on the same data with the same `random_state=42` seed, so accuracy should be identical.

Q: Can I add my own data?

A: Yes. Replace the CSVs in the `data/` folder with your own data (matching the schema), then run `python data/generate_data.py` to regenerate the SQLite DB. All charts and queries will automatically use your data.

Q: Why is navigation a selectbox and not clickable links?

A: Streamlit's native multipage navigation (using a `pages/` folder) creates a sidebar with automatic links but limits customisation. The selectbox approach gives full control over styling, programmatic navigation, and the single-file architecture. The tradeoff is that navigation requires opening a dropdown.

Q: How is the app deployed for free?

A: Streamlit Community Cloud offers free hosting for public GitHub repositories. The app auto-deploys on every git push to main. The only cost is the Anthropic API key for AI features.

Q: What is the Composite Risk Score?

A: A weighted point system across 5 customer health signals: health score, ticket volume, usage sessions, onboarding completion, and NPS. Maximum possible score is 100. Score ≥ 50 triggers a high-risk alert. Score ≥ 80 is critical. This mirrors how real CSM platforms like Gainsight calculate customer health.

Q: Can I use this project as a template for my own portfolio?

A: Yes. Fork the GitHub repository, replace the business scenario with your own, update the synthetic data schema, and redeploy. All the Streamlit patterns, CSS system, and AI integration code are reusable.

You now know BridgeIQ

inside out, front to back, top to bottom.

Built by Sai Hemanth

gtrhemanth14@gmail.com

github.com/gtrhemanth/BridgeIQ-

bridgeiq.streamlit.app